

Heap Sort

O **Heap Sort** é um algoritmo de ordenação que utiliza uma estrutura chamada **Heap (Monte)**, representada por uma **árvore binária completa**

Heap Sort

Imagine que você organiza números em uma árvore onde o maior valor sempre fica no topo (raiz). Essa estrutura é chamada de **Max-Heap**.

Heap Sort

Exemplo de regra do Max-Heap:

Pai $\geq$ Filho esquerdo

Pai $\geq$ Filho direito

Heap Sort

Depois de construir essa árvore, o algoritmo:

- Coloca o maior elemento na raiz.
- Troca a raiz pelo último elemento.
- Remove o maior elemento da árvore.
- Reconstrói o Heap.
- Repete até que todos os elementos estejam ordenados.

Heap Sort

Vetor inicial

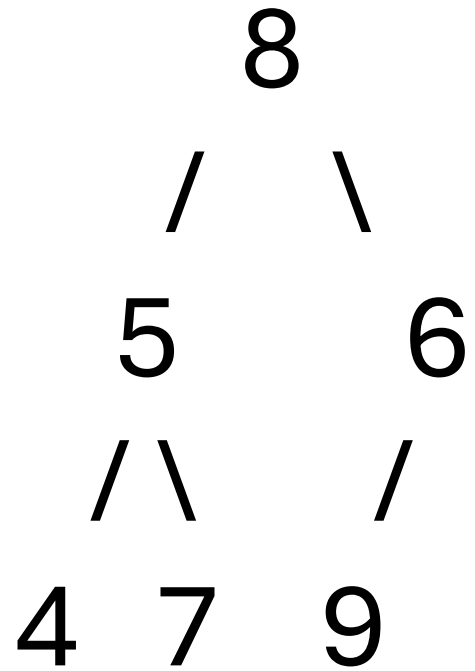
[8, 5, 6, 4, 7, 9]

Passo 1 - Representar como árvore binária

Em um Heap, os índices do vetor representam posições na árvore.

Heap Sort

Árvore:



Heap Sort

Observe que ainda NÃO é um Max-Heap porque:

$$9 > 6$$

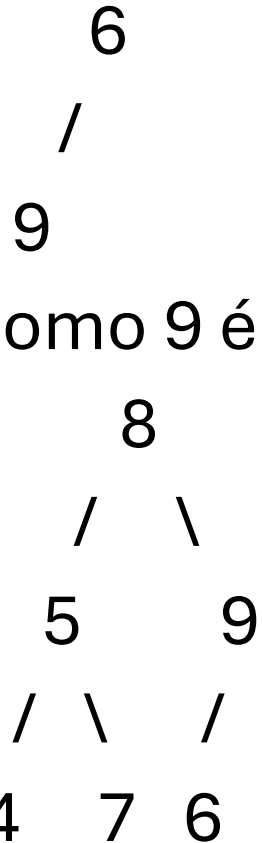
O pai deveria ser maior que os filhos.

Heap Sort

Passo 2 - Construir o Max-Heap

Começamos ajustando os nós de baixo para cima.

Verificando o nó 6



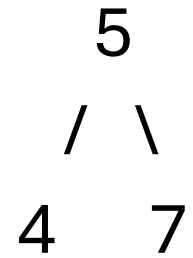
Como 9 é maior que 6, trocamos:

Heap Sort

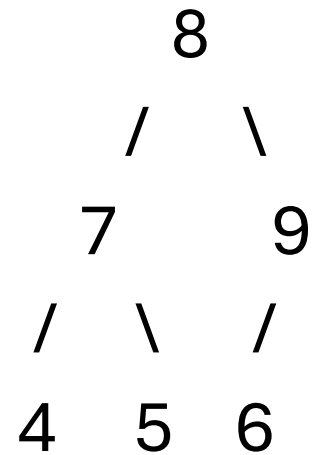
Vetor:

[8,5,9,4,7,6]

Verificando o nó 5



7 é maior que 5. Troca:

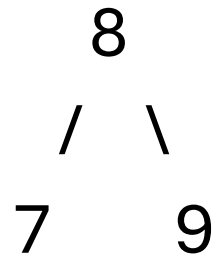


Heap Sort

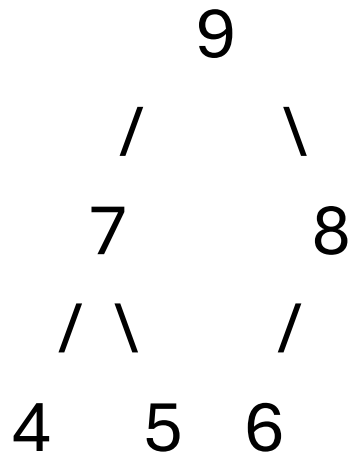
Vetor:

[8,7,9,4,5,6]

Verificando a raiz 8



9 é maior. Troca:



Heap Sort

Vetor:

[9,7,8,4,5,6]

Agora temos um **Max-Heap**.

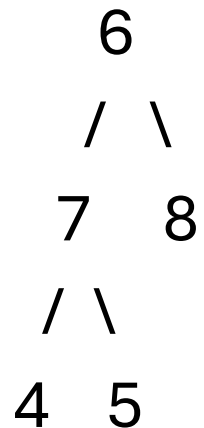
Passo 3 - Primeira ordenação

Maior elemento: 9

Troca com o último: [6,7,8,4,5,9]

9 já está na posição correta.

Árvore restante:

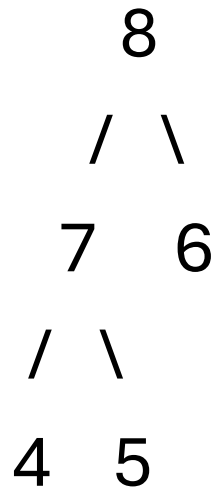


Heap Sort

Reorganizando o Heap

8 é o maior filho.

Troca:



Vetor:

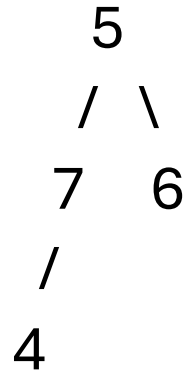
[8,7,6,4,5,9]

Heap Sort

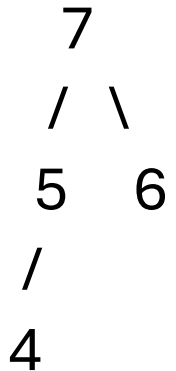
Passo 4 - Segunda ordenação

Troca raiz com último elemento válido: [5,7,6,4,8,9]

Árvore:



Reorganizando:



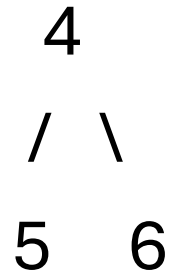
Vetor: [7,5,6,4,8,9]

Heap Sort

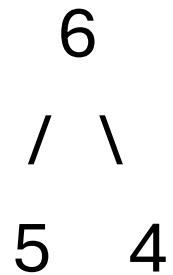
Passo 5 - Terceira ordenação

Troca: [4,5,6,7,8,9]

Árvore:



Reorganizando:



Vetor: [6,5,4,7,8,9]

Heap Sort

Passo 6 - Quarta ordenação

Troca: [4,5,6,7,8,9]

Reorganizando:

5

/

4

Vetor:

[5,4,6,7,8,9]

Heap Sort

Passo 7 - Última ordenação

Troca: [4,5,6,7,8,9]

Fim da ordenação.

Resultado final

[4,5,6,7,8,9]

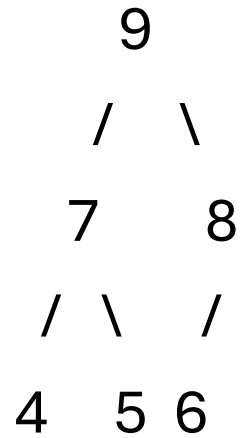
Heap Sort

Resumo visual

Vetor inicial

[8,5,6,4,7,9]

Max-Heap construído



Heap Sort

Extrações

9 → [8,7,6,4,5]

8 → [7,5,6,4]

7 → [6,5,4]

6 → [5,4]

5 → [4]

Vetor ordenado

[4,5,6,7,8,9]

Ideia principal para memorizar

O Heap Sort primeiro transforma o vetor em uma árvore onde o maior valor fica na raiz (Max-Heap). Depois ele remove repetidamente o maior elemento, colocando-o no final do vetor, até que todos os elementos estejam ordenados.

Heap Short

```
#include <stdio.h>
// Função para ajustar o Heap
void heapify(int vetor[], int n, int i){
    int maior = i;
    int esquerda = 2 * i + 1;
    int direita = 2 * i + 2;
    if (esquerda < n && vetor[esquerda] > vetor[maior])
        maior = esquerda;

    if (direita < n && vetor[direita] > vetor[maior])
        maior = direita;

    if (maior != i){
        int temp = vetor[i];
        vetor[i] = vetor[maior];
        vetor[maior] = temp;

        heapify(vetor, n, maior);
    }
}
```

Heap Short

```
// Função Heap Sort
void heapSort(int vetor[], int n)
{
    // Construção do Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(vetor, n, i);

    // Extração dos elementos
    for (int i = n - 1; i > 0; i--)
    {
        int temp = vetor[0];
        vetor[0] = vetor[i];
        vetor[i] = temp;

        heapify(vetor, i, 0);
    }
}
```

Heap Short

```
// Impressão do vetor
void imprimir(int vetor[], int n){
    for (int i = 0; i < n; i++)
        printf("%d ", vetor[i]);

    printf("\n");
}

int main(){
    int vetor[] = {8, 5, 6, 4, 7, 9};
    int n = sizeof(vetor) / sizeof(vetor[0]);
    printf("Vetor original:\n");
    imprimir(vetor, n);
    heapSort(vetor, n);
    printf("Vetor ordenado:\n");
    imprimir(vetor, n);
    return 0;
}
```